UNIVERSIDAD TECNOLÓGICA DE PANAMÁ
FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

MATERIAL DIDÁCTICO

INGENIERIA DE SOFTWARE III

MÓDULO IV
PRUEBAS UNITARIAS Y DE INTEGRACIÓN



DOCENTE/FACILITADOR:
Ramón G. González Tribaldos
ramon.gonzalez@utp.ac.pa

UTPCH - II Semestre 2026

Contenido Módulo IV

01 Definición de prueba unitaria

02 System Under Test (SUT) y Depended On Component

03 Test Doubles

04 Pruebas unitarias con Pytest y unittest.mock

05 Pruebas de Integración

06



01

Definición de prueba unitaria

¿Qué es una prueba unitaria?

- Una **prueba unitaria** es una prueba automatizada que verifica el comportamiento de la unidad de código más pequeña que tiene sentido probar (una función, un método o una clase) de forma aislada.
- También se le conoce como **prueba de componente** (ISTQB).
- Detecta defectos temprano, pero no demuestra que el sistema completo funcione: que cada pieza funcione sola no garantiza que funcionen juntas.

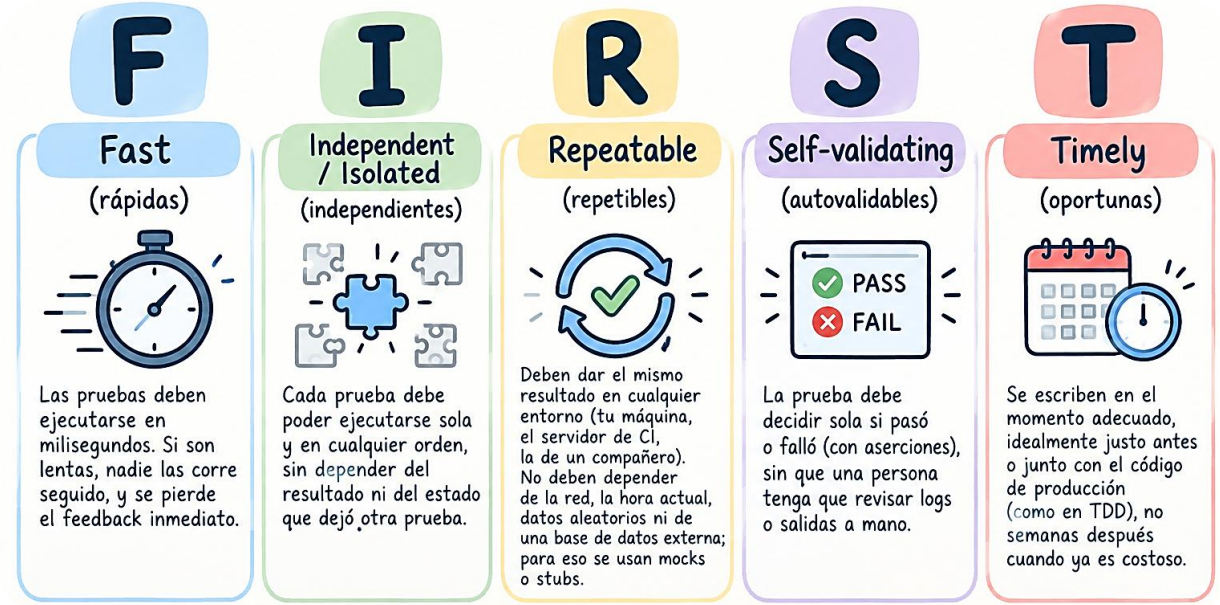


LO QUE NO ES UNA PRUEBA UNITARIA

“Cuando una prueba toca una base de datos real, la red, un sistema de archivos o el reloj del sistema, ya no es unitaria; es de integración”.

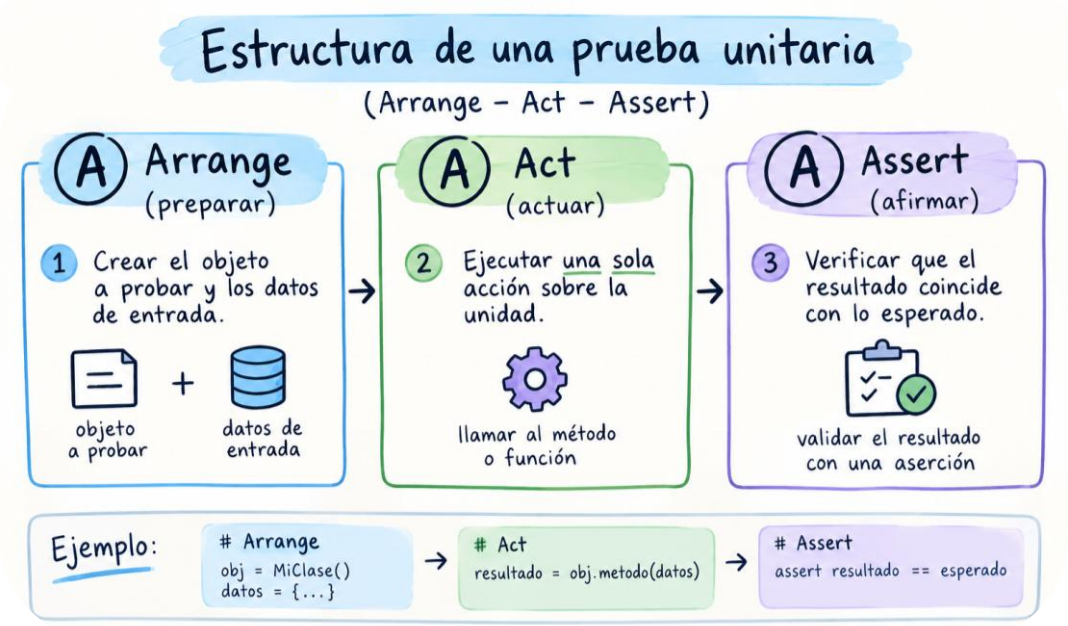
¿Qué es una prueba unitaria?

Características de una prueba unitaria



Estructura de una prueba unitaria

Patrón AAA (Arrange – Act – Assert)



Estructura de una prueba unitaria

Patrón AAA (Arrange – Act – Assert)

- Estructura estándar para organizar una prueba unitaria en tres fases:
  - **Arrange (preparar):** dejar listo todo lo que la prueba necesita, datos de entrada, el objeto a probar (si lo hay) y sus dependencias en el estado inicial.
  - **Act (actuar):** ejecutar una sola acción sobre la unidad bajo prueba (llamar a la función o método) y guardar su resultado.
  - **Assert (afirmar):** verificar que el resultado coincide con lo esperado (valor retornado, cambio de estado o error lanzado).



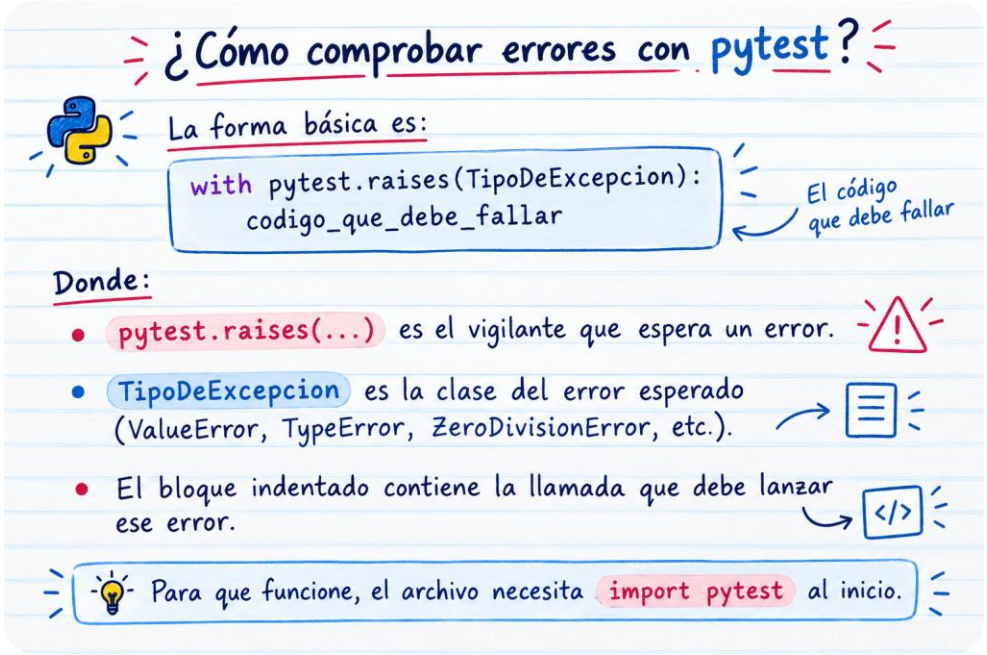
Estructura de una prueba unitaria

Ejemplo: Patrón AAA (Arrange – Act – Assert)

```
    def test_calc_total_itbms():
        # Arrange
        subtotal = 100.00
        # Act
        total = calcular_total(subtotal)
        # Assert
        assert total == 107.00
    def test_calc_total_negativo():
        with pytest.raises(ValueError):
            calcular_total(-5)
```

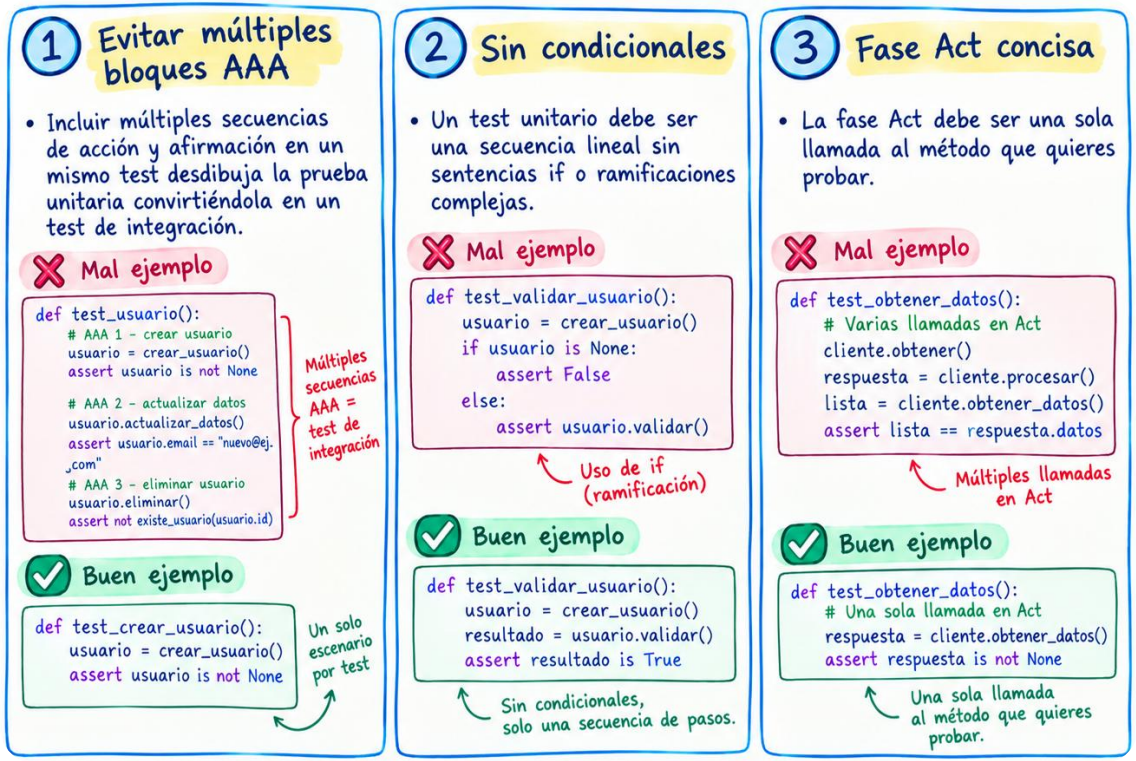


Estructura de una prueba unitaria



Permite comprobar que un código **rechaza correctamente** lo que no debe aceptar.

Reglas de Diseño Estructural



Ejercicio 01

Ejercicio 1: Total con ITBMS

Contexto

Una tienda en Panamá calcula el total de una compra sumando el 7 % de ITBMS al subtotal.



Código dado

```python
def calcular_total(subtotal):
    return subtotal * 1.07
```

Tarea

Escribir 2 pruebas con la estructura AAA.

Casos de prueba

1. Con un subtotal de 100, el total es 107.
2. Con un subtotal de 0, el total es 0.

Ejercicio 02

Ejercicio 2: Promedio de tres notas

Contexto:

Un estudiante quiere saber su promedio de tres evaluaciones.



Código dado:

```python
def calcular_promedio(nota1, nota2, nota3):
    return (nota1 + nota2 + nota3) / 3
```

Tarea: escribir 2 pruebas con la estructura AAA.

Casos de prueba

1. Con las notas 80, 90 y 100, el promedio es 90.
2. Con las notas 70, 70 y 70, el promedio es 70.



Ejercicio 03

Ejercicio 3: Depósito en una cuenta de ahorros

Contexto:

Un cliente deposita dinero y quiere saber cuánto tiene en su cuenta después del depósito.



Código dado:

```python
def depositar(saldo, monto):
    return saldo + monto
```

Tarea: escribir 2 pruebas con la estructura AAA.

Casos de prueba

1. Con un saldo de 100 y un depósito de 50, el nuevo saldo es 150.
2. Con un saldo de 0 y un depósito de 25, el nuevo saldo es 25.

02

System Under Test (SUT)
Depended On Component

System Under Test | Depended-On Component



System Under Test (SUT)

Es lo que estás probando en esa prueba. Es el objeto o la función cuyo comportamiento quieres verificar.



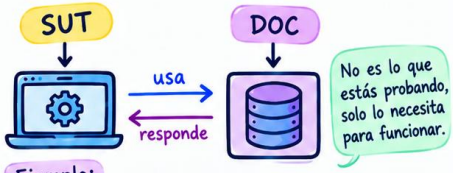
Ejemplo:

```
class Calculadora:
    def sumar(self, a, b):
        return a + b
```

Aquí, la clase Calculadora es el SUT (la estamos probando).

Depended-On Component (DOC)

Es algo que el SUT necesita para funcionar, pero que no es lo que estás probando. Puede ser otra clase, un servicio de correo, una base de datos, la hora del sistema, una API externa, etc.



Ejemplo:

```
class Cuenta:
    def __init__(self, saldo, notificador):
        self.saldo = saldo
        self.notificador = notificador  # <- DOC

    def retirar(self, monto):  # <- método del SUT
        self.saldo -= monto
        self.notificador.enviar("Retiro realizado")
```

El SUT es la clase Cuenta (método retirar).
El DOC es el notificador (por ejemplo, un servicio de correo).

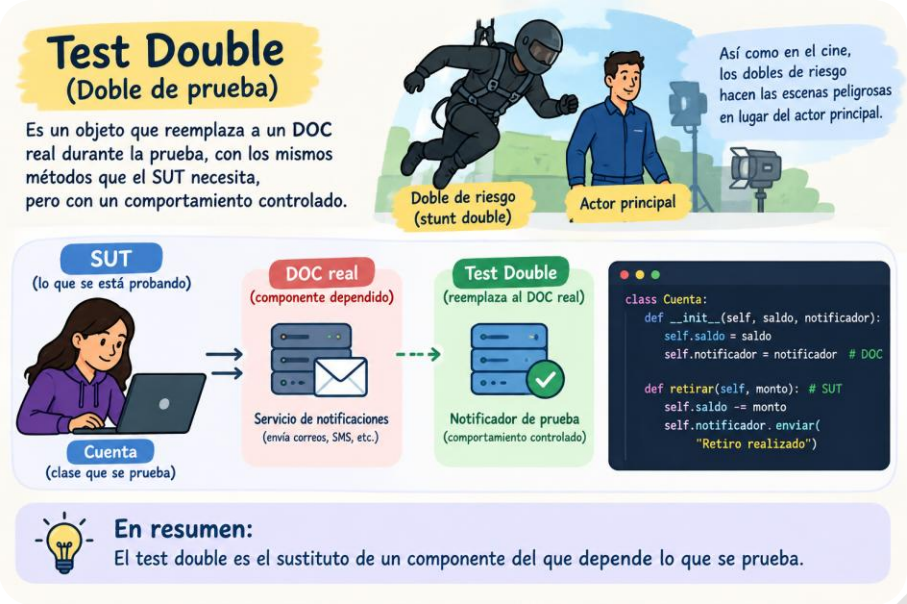
En resumen: SUT = lo que pruebas DOC = lo que necesita, pero no pruebas

03

Test Doubles

¿Qué es test double?

- Un **test double** (doble de prueba) es un objeto que reemplaza a un DOC real durante la prueba, con los mismos métodos que el SUT necesita, pero con un comportamiento controlado. El nombre viene de los dobles de riesgo en el cine.
- Es el sustituto de un componente del que depende lo que se prueba: es un objeto falso que reemplaza dependencias (BD, correos, API).



Tipos de test double

Dummy

Se usa cuando el componente solo se necesita para cumplir con la firma del método, pero su comportamiento no importa.



No hace nada, solo existe.

```
class DummyNotificador:
    def enviar(self, mensaje):
        pass # No hace nada
```

Stub

Devuelve valores predefinidos para que el SUT pueda continuar su ejecución. No se preocupa por registrar llamadas ni verificar el comportamiento.



Siempre devuelve lo mismo.

```
class StubServicio:
    def obtener_datos(self):
        return {"resultado": True}
```

Spy

Registra las llamadas que se hacen al objeto (spy), y puede además devolver valores predefinidos. Se usa para verificar que se interactúa correctamento con él.



Observa y registra lo que sucede.

```
class SpyServicio:
    def obtener_datos(self):
        self.llamadas.append("obtener_datos")
        return {"resultado": True}
```

Mock

Se usa para definir expectativas sobre cómo debe comportarse el objeto. Verifica que se llamen ciertos métodos (con parámetros específicos, en un orden, veces, etc.).



Verifica que se cumplan las expectativas.

```
class MockServicio:
    def obtener_datos(self):
        # Verifica que se llamó
        # con los parámetros esperados
        pass
```

Fake

Es una implementación real, pero simplificada o en memoria del componente. Se usa cuando quieres un comportamiento más cercano al real, pero sin depender de sistemas externos (ike una base de datos, una API, etc.).



Una versión simple del componente real.

```
class FakeBaseDatos:
    def obtener_datos(self):
        return ["dato1", "dato2"]
```

04

Pruebas unitarias con Pytest y unittest.mock

Pruebas con unittest.mock

¿Qué es unittest.mock?

`unittest.mock` es un módulo de la biblioteca estándar de Python que permite crear objetos de prueba (mocks, stubs, etc.) para simular el comportamiento de los componentes que un SUT (System Under Test) necesita, sin depender de su implementación real.



¿Para qué se usa?

- ✓ Aislar el SUT de sus dependencias.
- ✓ Controlar el comportamiento de los componentes simulados.
- ✓ Verificar que se llamen los métodos esperados (interacciones).
- ✓ Hacer las pruebas más rápidas, confiables y deterministas.

Principales herramientas de unittest.mock

Mock

Es el objeto principal. Permite comprobar llamadas y configurar su comportamiento.

```
from unittest.mock import Mock

m = Mock()
m.saludar()
m.assert_called_once()
```

MagicMock

Igual que Mock, pero incluye métodos mágicos (como __len__, __iter__, etc.).

```
from unittest.mock import MagicMock

mm = MagicMock()
len(mm)  # 0 por defecto
mm.__iter__()  # iterable
```

patch

Reemplaza temporalmente un objeto (por ejemplo, una función o clase) en el módulo donde se usa.

```
from unittest.mock import patch

@patch('mi_modulo.EmailService')
def test_enviar_email(mock_email):
    mock_email.send.assert_called_once()
```

side_effect

Permite definir un comportamiento personalizado (una función, una lista de valores o una excepción).

```
mock = Mock()
mock.obtener_datos.side_effect = [
    {'id': 1}, {'id': 2}
]
```

↳ La llamada 1 devuelve el primer valor, la llamada 2 el segundo.

Pruebas con unittest.mock

unittest.mock: Crear el mock

Con unittest.mock puedes crear **objetos falsos (mocks)** que imitan el comportamiento de las dependencias de tu código, sin necesidad de usar las implementaciones reales.

Un mock es un objeto que simula el comportamiento de algo real. Te permite probar tu código sin depender de servicios o componentes externos.

1 Crear el mock

```
m = Mock()
```

Crea un objeto falso (mock).
Este objeto imita el comportamiento de una dependencia.

```
m("hola")
```

Puedes llamar al mock como si fuera una función.
Esto es útil cuando el código espera que la dependencia sea llamable (callable).

```
m.guardar("a", "b")
```

El mock acepta cualquier método que necesites.
Si llamas a un método que no existe, lo crea automáticamente y lo proporciona para que puedas usarlo en tu prueba.

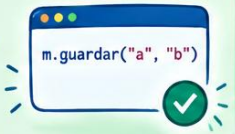
2 m("hola")

El mock se puede llamar como una función. No importa qué argumentos le pases, él los acepta.



3 m.guardar("a", "b")

El mock puede tener cualquier método que necesites. Si el método no existe, lo crea automáticamente y lo pone a tu disposición.



¿Y MagicMock?

MagicMock es un tipo especial de Mock que además soporta operaciones de Python como len(), for, with, [].

Empieza siempre con Mock y usa MagicMock solo si tu código lo necesita.

Pruebas con unittest.mock

unittest.mock: Configurar respuestas

Puedes indicar qué respuesta debe dar el mock cuando es llamado, o qué excepción debe lanzar.

Con unittest.mock puedes definir cómo responde un mock a las llamadas que hace tu código. Esto te permite simular diferentes comportamientos del componente dependiente (por ejemplo, una base de datos, un servicio externo, etc.).

Configuración	Qué hace	Ejemplo visual
`Mock(return_value=False)`	**Siempre devuelve el mismo valor** Cada vez que llames al mock, obtendrás el mismo resultado.	Mock → False Siempre False
`Mock(side_effect=ConnectionError("BD caída"))`	**Lanza una excepción** Simula un error o fallo en el componente dependiente.	Mock → ConnectionError ("BD caída") Simula un fallo del servicio
`Mock(side_effect=[False, True])`	**Devuelve valores diferentes por llamada** Cada llamada al mock devuelve el siguiente valor de la lista.	Mock → False → True → ... Útil para probar flujos que cambian en el tiempo.
`Mock(side_effect=lambda c: c == "4-123-456")`	**Decide según una función** La función recibe los argumentos de la llamada y devuelve el valor que corresponda.	Mock → True / False (según la lógica) Muy útil cuando la respuesta depende de los argumentos.

Regla práctica

Usa `return_value` cuando la respuesta es siempre la misma.

Usa `side_effect` cuando la respuesta cambia o necesitas simular un fallo.

En resumen

- `return_value` → respuesta fija (siempre igual).
- `side_effect` → respuesta variable, lista, excepción o función.
- Con estas opciones puedes simular fácilmente el comportamiento de tus dependencias en las pruebas.



Pruebas con unittest.mock

unittest: Verificar cómo se usó

Las aserciones de unittest te permiten comprobar que las funciones se llamaron la cantidad correcta de veces y con los argumentos esperados.

Estas aserciones son muy útiles cuando usas Mocks para verificar la interacción con otras clases o servicios.

Aserción / atributo	¿Qué comprueba?
`assert_called()`	Verifica que el método fue llamado **al menos una vez.**
`assert_called_once()`	Verifica que el método fue llamado **exactamente una vez.**
`assert_called_with(...)`	Verifica que el método fue llamado **con los argumentos esperados.**
`assert_called_once_with(...)`	Verifica que el método fue llamado **una sola vez y con los argumentos indicados.**
`assert_not_called()`	Verifica que el método **nunca fue llamado.**
`call_count`	Devuelve el **número de veces** que fue llamado el método (un entero).



En resumen

Estas aserciones te ayudan a verificar que tu código llamó correctamente a los métodos de sus dependencias.

Recuerda:

- Usa las aserciones que necesites según lo que quieras comprobar.
- Combinarlas te da pruebas más claras y confiables.

¡Así tus pruebas no solo verifican resultados, también la interacción entre componentes!

Pruebas con unittest.mock

Análisis detallado de ejemplo práctico:

test_validador8.py

```python
import pytest
from unittest.mock import Mock
from validador8 import registrar_estudiante

def test_registra_estudiante():
    # Arrange
    existe = Mock(return_value=False)  # siempre responde falso
    guardar = Mock()

    # Act
    resultado = registrar_estudiante(
        "4-123-456", "ramon tribaldos", existe, guardar
    )

    # Assert
    assert resultado is True
    existe.assert_called_once_with("4-123-456")
    guardar.assert_called_once_with("4-123-456", "ramon tribaldos")
```

validador8.py

```python
import re

def formato_valido(cedula):
    return re.fullmatch(r"\d{1,2}-\d{1,4}-\d{1,6}", cedula)
           is not None

def registrar_estudiante(cedula, nombre, existe_cedula, guardar):
    if not formato_valido(cedula):
        raise ValueError("Cédula con formato inválido")

    if existe_cedula(cedula):   # DOC 1: consulta a BD
        raise ValueError("La cédula ya está registrada")

    guardar(cedula, nombre)     # DOC 2: escritura en BD
    return True
```

¿Cómo se ejecuta esta prueba unitaria?

Recorrido visual: desde la preparación de los mocks hasta las comprobaciones de pytest.

OBJETIVO DE LA PRUEBA

Registrar un estudiante cuando su cédula es válida y todavía no existe en el sistema.



ARRANGE

Preparar los dobles

ACT

Ejecutar la función

ASSERT

Verificar el resultado

1. Arrange: preparar el escenario

En esta fase se crean los dobles. Todavía no se ejecuta registrar_estudiante().

```
existe = Mock(return_value=False)
guardar = Mock()

# No consultan ni escriben en una BD real
# Los mocks permiten controlar y observar
# las interacciones de la función.
```

Mock: existe

Cuando lo llamen, responderá siempre False: la cédula no existe.

Mock: guardar

No guarda realmente en una base de datos; registra que fue llamado y con qué argumentos.

Idea clave: los DOC se sustituyen por objetos controlados durante la prueba.

2. Act: ejecutar el método bajo prueba

La fase Act contiene una sola llamada: la función que queremos probar.

```
resultado = registrar_estudiante(
    "4-123-456", "ramon tribaldos", existe, guardar
)
```

¿Qué recibe la función?

Cédula • Nombre • Mock de consulta • Mock de guardado

A partir de aquí, seguimos el flujo interno de registrar_estudiante().

3. Primera comprobación: formato de la cédula

La función valida la entrada antes de consultar si la cédula ya existe.

```
def registrar_estudiante(...):
    if not formato_valido(cedula):
        raise ValueError("...")


def formato_valido(cedula):
    return re.fullmatch(
        r"\d{1,2}-\d{1,4}-\d{1,6}",
        cedula
    ) is not None
```

Cédula recibida

4-123-456

Coincide con el patrón

Formato válido → continúa

Como el formato es válido, NO se lanza ValueError.

4. Segunda comprobación: ¿la cédula ya existe?

Se ejecuta la dependencia DOC 1 mediante el Mock existe.

```
if existe_cedula(cedula):
    raise ValueError("Ya registrada")
```

Llamada realizada

existe("4-123-456")

El Mock responde: False

False significa que la cédula NO está registrada. La condición no se cumple y la función continúa.

El Mock permite simular la respuesta de la consulta sin acceder a una BD real.

5. Se solicita guardar el estudiante

Como la cédula es válida y no existe, la función llega a la segunda dependencia.

```
guardar(cedula, nombre)
return True
```

Llamada al Mock guardar

```
guardar(
    "4-123-456",
    "ramon tribaldos")
```

Después de llamar a guardar(...), la función devuelve True.

Importante: el Mock registra la llamada; no escribe realmente en la base de datos.

6. Assert: comprobar el resultado

Ahora pytest comprueba si el comportamiento observado coincide con lo esperado.

```
assert resultado is True


existe.assert_called_once_with("4-123-456")


guardar.assert_called_once_with(
    "4-123-456", "ramon tribaldos"
)
```

Resultado

True ✓

Llamadas verificadas

existe: 1 vez ✓

guardar: 1 vez ✓

Las llamadas usaron los argumentos esperados.

Resumen: el recorrido de la prueba

El caso exitoso queda demostrado por el resultado y por las interacciones con los DOC.

1	**Arrange**	Se preparan los mocks
2	**Act**	Se llama a registrar_estudiante(...)
3	**Validación**	El formato de la cédula es válido
4	**Consulta**	existe(...) responde False
5	**Guardado**	guardar(...) recibe cédula y nombre
6	**Assert**	pytest verifica resultado y llamadas

Prueba superada: la función registró correctamente al estudiante en el escenario simulado.

Pruebas con unittest.mock

Práctica: Función realizar_compra()

La función realizar_compra() verifica si hay existencias del producto antes de guardar la compra. Usa funciones como parámetros para simular las dependencias externas.

Esta función recibe otras funciones como argumentos para poder simular sus dependencias en las pruebas.

Código base

Producto

Función para verificar stock

```
def realizar_compra(producto, verificar_stock, guardar):
    # Consultar si hay existencias
    disponible = verificar_stock(producto)

    if not disponible:
        return False

    # Guardar la compra
    guardar(producto)

    return True
```

Función para guardar la compra

¿Cómo funciona?

1. **Consulta si hay existencias**
   Llama a la función verificar_stock() con el producto.
   → verificar_stock (producto)

2. **Si no hay existencias**
   Devuelve False y no llama a guardar().
   → False

3. **Si hay existencias**
   Llama a guardar() con el producto y devuelve True.
   → guardar (producto) → True

Pruebas con unittest.mock

Práctica: Probar con mocks

La función realizar_compra() verifica si hay existencias del producto antes de guardar la compra. Usa mocks para simular las dependencias y comprobar su comportamiento.

Recuerda:
Un mock te permite simular una función o dependencia, definir qué devuelve y verificar cómo fue llamada.

1 Compra exitosa

Configura un Mock para que verificar_stock devuelva `True`. Comprueba que la función devuelve `True` y que guardar se llama una sola vez con el producto correcto.



Lo que debes comprobar:

- Que la función devuelve `True`.
- Que verificar_stock se llamó una sola vez con `"Teclado"`.
- Que guardar se llamó una sola vez con `"Teclado"`.

2 Producto sin existencias

Configura un Mock para que verificar_stock devuelva `False`. Comprueba que la función devuelve `False` y que guardar no se llama.



Lo que debes comprobar:

- Que la función devuelve `False`.
- Que verificar_stock se llamó una sola vez con `"Teclado"`.
- Que guardar no se llamó.



CURSO DE INGENIERÍA DE SOFTWARE III.

Licenciatura en Desarrollo y Gestión de Software. FISC 2026.